## Source Code Disclosure

Project Name: ReelBite: Cinematic Cravings

Author: Mr. Pranav Gupta and Dr. Rajat Takkar

Category: Computer Software
Languages: JavaScript,Express.js, Node.js, React ,MongoDB,Tailwind CSS Date of Creation: 2026

This document contains partial source code submitted for copyright registration under the Copyright Act, 1957 (India).

## ReelBite-Cinematic-Cravings

Welcome to "Reel Bite: Cinematic Cravings": It is a cutting-edge ecommerce online platform catering to movie aficionados and food enthusiasts alike. Leveraging React and JavaScript frameworks, our website provides a seamless interface for accessing the latest blockbuster movie reviews, booking tickets, and ordering meals and combos directly.

## Outcomes & Benefits

The system significantly enhances user convenience by reducing booking time and eliminating fragmented workflows. It helps cinemas streamline operations, minimize manual errors, improve customer engagement, and increase revenue through integrated services and promotional features.

### Project Structure 🏗️

Right Resource Fit/ - ├── frontend/
- └── backend/
- └── admin/

This section contains the contents of the repository's files.

< path="backend/controllers/cartController.js">

const asyncHandler = require('express-async-handler');

const Cart = require('../models/cartModel');

const Product = require('../models/moviesModel');

const Product1 = require('../models/foodModel');

const addToCart = asyncHandler(async (req, res) => { const { productId, quantity } = req.body; const userId = req.user ? req.user._id : null; const sessionId = req.sessionID; // For guest users let product = await Product.findById(productId); // If the product is not found in the Product model, try Product1 if (!product) { product = await

Product1.findById(productId); if (!product) { return res.status(400).json({ message: ‘Insufficient stock or product not available’ }); } } // Get or create a cart (based on userId or sessionId) let cart; if (userId) { cart = await Cart.findOne({ userId, isExpired: false }) || new Cart({ userId, items: [] }); } else { cart = await Cart.findOne({ sessionId, isExpired: false }) || new Cart({ sessionId, items: [] }); } // Check if the product already exists in the cart const itemIndex = cart.items.findIndex(item => item.productId.toString() === productId); if (itemIndex > -1) { // If it exists, update the quantity and total price const item = cart.items[itemIndex]; item.quantity += quantity; item.totalPrice = item.price * item.quantity; } else { // Otherwise, add a new product to the cart cart.items.push({ productId: product._id, productName: product.productName, image: product.image, productDescription:product.productDescription , price: product.productPrice, quantity, totalPrice: product.productPrice * quantity }); } // Calculate the total price const subTotal = cart.items.reduce((sum, item) => sum + item.totalPrice, 0); cart.totalPrice = subTotal;

const totalQuantity = cart.items.reduce((sum, item) => sum + item.quantity, 0); // Calculate total quantity cart.totalQuantity = totalQuantity; // Set cart expiration (e.g., 30 days) cart.expiresAt = new Date(Date.now() + 30 * 24 * 60 * 60 * 1000); // Save the cart await cart.save(); res.status(200).json(cart); });

const removeFromCart = asyncHandler(async (req, res) => { const { productId, quantity } = req.body; const userId = req.user ? req.user._id : null; const sessionId = req.sessionID; // For guest users if (!productId || quantity === undefined) { return res.status(400).json({ message: ‘Product ID and quantity are required’ }); } // Find product let product = await Product.findById(productId); if (!product) { product = await Product1.findById(productId); if (!product) { return res.status(400).json({ message: ‘Product not available’ }); } } // Find cart let cart; if (userId) { cart = await Cart.findOne({ userId, isExpired: false }); } else { return res.status(404).json({ message: ‘Cart not found or expired’ }); } // const cart = await Cart.findOne({ $or: [{ userId }, { sessionId }], isExpired: false }); // if (!cart) { // return res.status(404).json({ message: ‘Cart not found or expired’ }); // } // Find the item in the cart const itemIndex = cart.items.findIndex(item => item.productId.toString() === productId); if (itemIndex > -1) { const item = cart.items[itemIndex]; // Ensure that the requested quantity to remove does not exceed the current quantity if (quantity > item.quantity) { return res.status(400).json({ message: ‘Quantity to remove exceeds item quantity in cart’ }); } // Decrease the quantity or remove the item if necessary item.quantity -= quantity; if (item.quantity <= 0) { // Remove the item from the cart if the quantity becomes zero or less cart.items.splice(itemIndex, 1); } else { // Update the total price if the item remains in the cart item.totalPrice = item.price * item.quantity; } // Recalculate the cart’s total price const subTotal = cart.items.reduce((sum, item) => sum + item.totalPrice, 0); cart.totalPrice = subTotal; const totalQuantity = cart.items.reduce((sum, item) => sum + item.quantity, 0); cart.totalQuantity = totalQuantity; await cart.save(); return res.status(200).json(cart); } else { return res.status(404).json({ message: ‘Item not found in cart’ }); } });

const getCart = asyncHandler(async (req, res) => { const userId = req.user ? req.user._id : null; // const sessionId = req.sessionID; // Find cart const cart = await Cart.findOne({ userId , isExpired: false }); if (!cart || new Date() > cart.expiresAt) { return

res.status(404).json({ message: ‘Cart not found or expired’ }); } res.status(200).json(cart); });

const clearCart = asyncHandler(async (req, res) => { const userId = req.user._id; const cart = await Cart.findOne({ userId }); if (!cart) { return res.status(404).json({ message: ‘Cart not found’ }); } cart.items = []; cart.totalPrice = 0; cart.taxAmount = 0; cart.shippingCost = 0; cart.discountAmount = 0; cart.totalQuantity=0; await cart.save(); res.status(200).json({ message: ‘Cart cleared’ }); });

const addToCart1 = asyncHandler(async (req, res) => { const { productId, quantity } = req.body; const userId = req.user ? req.user._id : null; console.log(userId); // Check if logged in const sessionId = req.sessionID; // For guest users // Find product and validate stock const product = await Product1.findById(productId); if (!product ) { return res.status(400).json({ message: ‘Insufficient stock or product not available’ }); } // Get or create a cart (based on userId or sessionId) let cart; if (userId) { cart = await Cart.findOne({ userId, isExpired: false }) || new Cart({ userId, items: [] }); } else { cart = await Cart.findOne({ sessionId, isExpired: false }) || new Cart({ sessionId, items: [] }); } // Check if product already exists in cart const itemIndex = cart.items.findIndex(item => item.productId.toString() === productId); if (itemIndex > -1) { // Update quantity if product exists const item = cart.items[itemIndex]; item.quantity += quantity; item.totalPrice = item.price * item.quantity; } else { // Add new product to cart cart.items.push({ productId: product._id, productName: product.productName, image: product.image, price: product.productPrice, quantity, totalPrice: product.productPrice * quantity }); } // Calculate total price, discount, tax, and shipping const subTotal = cart.items.reduce((sum, item) => sum + item.totalPrice, 0); // cart.taxAmount = subTotal * 0.1; // Assuming 10% tax // cart.shippingCost = subTotal > 100 ? 0 : 10; // Free shipping for orders > $100 cart.totalPrice = subTotal ; const totalQuantity = cart.items.reduce((sum, item) => sum + item.quantity, 0); cart.totalQuantity = totalQuantity; // cart.totalPrice = subTotal + cart.taxAmount + cart.shippingCost - cart.discountAmount; // Set cart expiration (e.g., 30 days) cart.expiresAt = new Date(Date.now() + 30 * 24 * 60 * 60 * 1000); // Save cart await cart.save(); res.status(200).json(cart); });

const removeFromCart1 = asyncHandler(async (req, res) => {

const { productId } = req.body;

const userId = req.user ? req.user._id : null;

const sessionId = req.sessionID; // Find cart let cart; if (userId) { cart = await Cart.findOne({ userId, isExpired: false }); } else { return res.status(404).json({ message: ‘Cart not found or expired’ }); } // Remove item from cart cart.items = cart.items.filter(item => item.productId.toString() !== productId);

const totalQuantity = cart.items.reduce((sum, item) => sum + item.quantity, 0); cart.totalQuantity = totalQuantity; // Recalculate total

const subTotal = cart.items.reduce((sum, item) => sum + item.totalPrice, 0); // cart.taxAmount = subTotal * 0.1; cart.totalPrice = subTotal; await cart.save(); res.status(200).json(cart); });

const getCart1 = asyncHandler(async (req, res) => {

const userId = req.user ? req.user._id : null;

const sessionId = req.sessionID; // Find cart

const cart = await Cart.findOne({ $or: [{ userId }, { sessionId }], isExpired: false }).populate('items.productId'); if (!cart || new Date() > cart.expiresAt) { return res.status(404).json({ message: 'Cart not found or expired' }); } res.status(200).json(cart); });

const clearCart1 = asyncHandler(async (req, res) => {

const userId = req.user._id;

const cart = await Cart.findOne({ userId }); if (!cart) { return res.status(404).json({ message: 'Cart not found' }); } cart.items = []; cart.totalPrice = 0; cart.taxAmount = 0; cart.shippingCost = 0; cart.discountAmount = 0; cart.totalQuantity=0; await cart.save(); res.status(200).json({ message: 'Cart cleared' }); }); module.exports={ addToCart, removeFromCart, getCart, clearCart,addToCart1, removeFromCart1, getCart1, clearCart1 };

< path="backend/controllers/foodsController.js">

const asyncHandler = require("express-async-handler");

const allProductModel = require("../models/foodModel"); //@desc Get all Products //@route GET /api/allProducts

const getProducts = asyncHandler(async (req, res) => { try{ const products = await allProductModel.find({}); res.status(200).json(products); }catch (error) { console.error(error); res.status(500).json({ success: false, message: 'Error fetching products' }); } }); //@desc Create New Product //@route POST /api/allProducts //@access private

const createProducts = asyncHandler(async (req, res) => { console.log("The request body is:", req.body); const { image, productName, productPrice, available, } = req.body; if ( !image || !productName || !productPrice||!available ) { res.status(400); throw new Error("All Fields are mandatory!"); } try{ let product = await allProductModel.find({}); let id; if (product.length > 0) { let lastProduct = product.slice(-1)[0]; id = lastProduct.id + 1; } else { id = 1; } const products = await allProductModel.create({ products_id:id, image, productName, productPrice, available }); res.status(201).json({ success: true, products, });} catch (error) { console.error(error); res.status(500).json({ success: false, message: 'Error adding product' }); } }); //@desc GET Products by Id //@route GET /api/allProducts/:id //@access private

const getProductsById = asyncHandler(async (req, res) => { const products = await allProductModel.findById(req.params.id); // console.log(`Products fetched:`, products); if (!products) { // console.log(`Product not found: ${req.params.id}`); res.status(400); throw new Error("Product Not Found"); } res.status(200).json(products); }); //@desc Update Products by Id //@route PUT /api/allProducts/:id //@access private

const updateProductsById = asyncHandler(async (req, res) => { try { // Find the product by ID const products = await allProductModel.find({ product_id: req.body.product_id }); if (!products) { res.status(404); throw new Error("Product Not Found"); } // if (products.products_id !== req.body.products_id) { // res.status(403); // throw new Error( // "User don't have permission to update other Product Details" // ); // } // Update the product const updatedProduct = await allProductModel.findOneAndUpdate( { products_id: req.body.products_id }, req.body, { new: true } ); return res.status(200).json({ success: true, updatedProduct }); } catch (error) { console.error("Error updating product:", error); return res.status(500).json({ success: false, message: 'Error updating product' }); } }); //@desc Delete Products By Id //@route DELETE /api/allProducts/:id //@access/private

const deleteProductsById = asyncHandler(async (req, res) => { try{ const products = await allProductModel.find({ products_id: req.body.product_id }); if (!products) { res.status(404); throw new Error("Product Not Found"); } await allProductModel.deleteOne({ products_id: req.body.product_id }); res.status(200).json({ success: true, name: req.body.name, }); }catch(error){ console.error(error); res.status(500).json({ success: false, message: 'Error removing product' }); } }); module.exports = { getProducts, createProducts, getProductsById, updateProductsById, deleteProductsById, };

< path="backend/controllers/moviesController.js">

const asyncHandler = require("express-async-handler");

const allProductModel = require("../models/moviesModel"); //@desc Get all Products //@route GET /api/allProducts

const getProducts = asyncHandler(async (req, res) => { try{ const products = await allProductModel.find({}); res.status(200).json(products); }catch (error) { console.error(error); res.status(500).json({ success: false, message: 'Error fetching products' }); } }); //@desc Create New Product //@route POST /api/allProducts //@access private

const createProducts = asyncHandler(async (req, res) => { console.log("The request body is:", req.body); const { image, productName, productDescription, productPrice, category, available, } = req.body; if ( !category || !image || !productName || !productDescription || !productPrice||!available ) { res.status(400); throw new Error("All Fields are mandatory!"); } try{ let product = await allProductModel.find({}); let id; if (product.length > 0) { let lastProduct = product.slice(-1)[0]; id = lastProduct.id + 1; } else { id = 1; } const products = await allProductModel.create({ products_id:id, image, productName, productDescription, productPrice, category,available }); res.status(201).json({ success: true, products, });} catch (error) { console.error(error); res.status(500).json({ success: false, message: 'Error adding

product' }); } }); //@desc GET Products by Id //@route GET /api/allProducts/:id //@access private

const getProductsById = asyncHandler(async (req, res) => { const products = await allProductModel.findById(req.params.id); // console.log(`Product not found: ${req.params.id}`); if (!products) { res.status(400); throw new Error("Product Not Found"); } res.status(200).json(products); }); //@desc Update Products by Id //@route PUT /api/allProducts/:id //@access private

const updateProductsById = asyncHandler(async (req, res) => { try { // Find the product by ID const products = await allProductModel.find({ products_id: req.body.products_id }); if (!products) { res.status(404); throw new Error("Product Not Found"); } // if (products.products_id !== req.body.products_id) { // res.status(403); // throw new Error( // "User don't have permission to update other Product Details" // ); // } // Update the product const updatedProduct = await allProductModel.findOneAndUpdate( { products_id: req.body.products_id }, req.body, { new: true } ); return res.status(200).json({ success: true, updatedProduct }); } catch (error) { console.error("Error updating product:", error); return res.status(500).json({ success: false, message: 'Error updating product' }); } }); //@desc Delete Products By Id //@route DELETE /api/allProducts/:id //@access/private

const deleteProductsById = asyncHandler(async (req, res) => { try{ const products = await allProductModel.find({ products_id: req.body.product_id }); if (!products) { res.status(404); throw new Error("Product Not Found"); } console.log(products.products_id); await allProductModel.deleteOne({ products_id: req.body.product_id }); res.status(200).json({ success: true, name: req.body.name, }); }catch(error){ console.error(error); res.status(500).json({ success: false, message: 'Error removing product' }); } }); module.exports = { getProducts, createProducts, getProductsById, updateProductsById, deleteProductsById, };

< path="backend/controllers/OrdersDetailsControllers.js">

const Order = require('../models/OrdersDetailModel'); // Create a new order

const createOrder = async (req, res) => { try { const { paymentSource,payment_status,payment_date,order_summary,paymentID,billing_information,email_address,national_number,} = req.body; const newOrder = new Order({ userId: req.user._id, order_summary, paymentID:paymentID, billing_address:billing_information, email_address:email_address, paymentSource: paymentSource, national_number:national_number, payment_status:payment_status, payment_date:payment_date, }); const savedOrder = await newOrder.save(); res.status(201).json({ message: 'Order placed successfully', order: savedOrder }); } catch (error) { res.status(500).json({ error: 'Failed to place order', details: error.message }); } }; // Get all orders for admin

const getAllOrders = async (req, res) => { try { const orders = await Order.find().populate('userId', 'name email').sort({ createdAt: -1 }); return res.status(200).json(orders); } catch (error) { return res.status(500).json({ error: 'Failed to fetch orders' }); } };

const getOrders = async (req, res) => { try { const userId = req.user._id; // Assuming userId is in req.user after authentication const orders = await Order.find({ userId: userId }); res.status(200).json(orders); } catch (error) { res.status(500).json({ error: 'Failed to fetch orders' }); } }; // Get a single order by ID for a user

const getOrderById = async (req, res) => { try { const order = await Order.findOne({ _id: req.params.orderId, userId: req.user._id }); if (!order) { return res.status(404).json({ message: 'Order not found' }); } res.status(200).json(order); } catch (error) { res.status(500).json({ error: 'Failed to fetch the order', details: error.message }); } }; // Update order status

const updateOrderStatus = async (req, res) => { const { status } = req.body; try { const order = await Order.findById(req.params.orderId); if (!order) { return res.status(404).json({ message: 'Order not found' }); } order.order_history.push({ status, updated_at: Date.now(), remarks: req.body.remarks }); order.order_summary.status = status; await order.save(); res.status(200).json({ message: 'Order status updated successfully', order }); } catch (error) { res.status(500).json({ error: 'Failed to update order status', details: error.message }); } }; // Cancel an order

const cancelOrder = async (req, res) => { try { const order = await Order.findOne({ _id: req.params.orderId, userId: req.user._id }); if (!order) { return res.status(404).json({ message: 'Order not found' }); } if (order.order_summary.status !== 'Pending') { return res.status(400).json({ message: 'Only pending orders can be canceled' }); } order.order_summary.status = 'Cancelled'; order.order_history.push({ status: 'Cancelled', updated_at: Date.now(), remarks: 'Order cancelled by user' }); await order.save(); res.status(200).json({ message: 'Order cancelled successfully', order }); } catch (error) { res.status(500).json({ error: 'Failed to cancel the order', details: error.message }); } }; // Add payment information

const addPaymentInfo = async (req, res) => { try { const { payment_method, transaction_id, payment_status } = req.body; const order = await Order.findOne({ _id: req.params.orderId, userId: req.user._id }); if (!order) { return res.status(404).json({ message: 'Order not found' }); } order.order_summary.payment_status = payment_status || 'Pending'; order.order_summary.payment_history.push({ payment_method, transaction_id, payment_status, payment_date: Date.now() }); await order.save(); res.status(200).json({ message: 'Payment added successfully', order }); } catch (error) { res.status(500).json({ error: 'Failed to add payment', details: error.message }); } }; // Update shipping details

const updateShippingDetails = async (req, res) => { try { const { shipping_method, tracking_number, estimated_delivery_date } = req.body; const order = await Order.findById(req.params.orderId); if (!order) { return res.status(404).json({ message: 'Order not found' }); } order.shipping_information.shipping_method = shipping_method; order.shipping_information.tracking_number = tracking_number; order.shipping_information.estimated_delivery_date = estimated_delivery_date; order.order_history.push({ status: 'Shipped', updated_at: Date.now(), remarks: 'Shipping details updated' }); await order.save(); res.status(200).json({ message: 'Shipping details

updated successfully', order }); } catch (error) { res.status(500).json({ error: 'Failed to update shipping details', details: error.message }); } }; // Process refund

const processRefund = async (req, res) => { const { refund_status, refund_amount } = req.body; try { const order = await Order.findById(req.params.orderId); if (!order) { return res.status(404).json({ message: 'Order not found' }); } order.refund_information.refund_status = refund_status || 'Requested'; order.refund_information.refund_amount = refund_amount; order.refund_information.refund_date = Date.now(); order.order_summary.payment_status = 'Refunded'; order.order_history.push({ status: 'Refunded', updated_at: Date.now(), remarks: `Refund of $${refund_amount} processed` }); await order.save(); res.status(200).json({ message: 'Refund processed successfully', order }); } catch (error) { res.status(500).json({ error: 'Failed to process refund', details: error.message }); } }; module.exports = { createOrder, getAllOrders, getOrderById, updateOrderStatus, cancelOrder, addPaymentInfo, updateShippingDetails, processRefund, getOrders };

< path="backend/controllers/userController.js">

const asyncHandler = require("express-async-handler");

const bcrypt = require("bcrypt");

const jwt = require("jsonwebtoken");

const User = require("../models/userModel");

const { sendPasswordResetEmail } = require("../utils/sendEmail");

const crypto = require("crypto"); require('dotenv').config(); // @desc Register a new user // @route POST /api/user/register // @access Public

const registerUser = asyncHandler(async (req, res) => { const { username, email, password } = req.body; // Check if all fields are provided if (!username || !email || !password) { return res.status(400).json({ success: false, error: "Please provide all fields" }); } // Check if user already exists (including deleted accounts) const userExists = await User.findOne({ email }); if (userExists) { if (userExists.isDeleted) { // Reactivate the deleted account userExists.username = username; userExists.password = await bcrypt.hash(password, await bcrypt.genSalt(10)); userExists.isDeleted = false; // Mark the user as not deleted await userExists.save(); // Generate JWT const token = jwt.sign({ id: userExists._id }, process.env.JWT_SECRET, { expiresIn: "1d", }); return res.status(200).json({ _id: userExists._id, username: userExists.username, email: userExists.email, token, }); } else { return res.status(400).json({ success: false, error: "Existing user found with the same email address", }); } } // Hash the password const salt = await bcrypt.genSalt(10); const hashedPassword = await bcrypt.hash(password, salt); // Create the user const user = await User.create({ username, email, password: hashedPassword, }); if (user) { // Generate JWT const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: "1d", }); return res.status(201).json({ _id: user._id, username: user.username, email: user.email, token, }); } else { return res.status(400).json({ success: false, error: "Invalid user data" }); } }); // @desc Authenticate user & get token // @route POST /api/user/login // @access Public

const loginUser = asyncHandler(async (req, res) => { const { email, password } = req.body; // Check if user exists const user = await User.findOne({ email }); if (user) { if (user.isDeleted) { return res.status(400).json({ success: false, error: “User account is deleted. Please contact support.” }); } if((await bcrypt.compare(password, user.password))){ // Generate JWT const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: “1d”, }); let role1; if (user.isAdmin){ role1=‘admin’ }else { role1=‘user’ } res.json({ _id: user._id, username: user.username, email: user.email, imageUrl: user.imageUrl, token, role:role1 }); }else{ res.status(400).json({success:false,error:“Wrong Password”}) } }else{ res.status(400).json({success:false,error:“Wrong email address”}) } }); // @desc Get user profile // @route GET /api/user/profile // @access Private

const getUserProfile = asyncHandler(async (req, res) => { const user = await User.findById(req.user.id); if (user) { let role1; if (user.isAdmin){ role1=‘admin’ }else { role1=‘user’ } res.json({ _id: user._id, username: user.username, email: user.email, imageUrl:user.imageUrl, role:role1 }); } else { res.status(404); throw new Error(“User not found”); } }); // @desc Update user profile // @route PUT /api/user/profile // @access Private

const updateUserProfile = asyncHandler(async (req, res) => { const user = await User.findById(req.user.id); if (user) { user.username = req.body.username || user.username; user.email = req.body.email || user.email; user.imageUrl = req.body.avatar || user.imageUrl; if (req.body.password) { const salt = await bcrypt.genSalt(10); user.password = await bcrypt.hash(req.body.password, salt); } const updatedUser = await user.save(); // Generate new JWT token const token = jwt.sign({ id: updatedUser._id }, process.env.JWT_SECRET, { expiresIn: “1d”, }); res.json({ _id: updatedUser._id, username: updatedUser.username, email: updatedUser.email, imageUrl: updatedUser.imageUrl, token, }); } else { res.status(404); throw new Error(“User not found”); } }); // @desc Request password reset link // @route POST /api/user/forgot-password // @access Public

const requestPasswordReset = asyncHandler(async (req, res) => { const { email } = req.body; if (!email) { res.status(400); throw new Error(“Please provide an email address”); } const user = await User.findOne({ email }); if (!user) { res.status(404); throw new Error(“User not found”); } if (user.isDeleted) { res.status(403); throw new Error(“Cannot reset password for a deleted user.”); } const resetToken = crypto.randomBytes(32).toString(“hex”); user.resetToken = resetToken; user.resetTokenExpire = Date.now() + 3600000; // 1 hour expiration await user.save(); const frontendUrl = req.headers.origin; // Get the request origin (domain) const resetUrl = `${frontendUrl}/reset-password/${resetToken}`; await sendPasswordResetEmail(user.email, resetUrl); res.json({ success: true }); }); // @desc Reset user password // @route POST /api/user/reset-password/:resetToken // @access Public

const resetPassword = asyncHandler(async (req, res) => { try { const { resetToken } = req.params; const { newPassword } = req.body; const user = await User.findOne({ resetToken, resetTokenExpire: { $gt: Date.now() }, }); if (!user) { return res.status(400).json({ success: false, message: ‘Invalid or expired token’ }); } if

(user.isDeleted) { return res.status(400).json({ success: false, message: 'User account is deleted. Please contact support.' }); } const salt = await bcrypt.genSalt(10); user.password = await bcrypt.hash(newPassword, salt); user.resetToken = undefined; // Clear reset token user.resetTokenExpire = undefined; // Clear token expiration await user.save(); const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d', }); res.json({ success: true, _id: user._id, username: user.username, email: user.email, token, }); } catch (error) { console.error('Error in resetting password:', error); // Log the actual error res.status(500).json({ success: false, message: 'Server error while resetting password' }); } }); // @desc Get all users (Admin only) // @route GET /api/user/all-users // @access Private/Admin

const getAllUsers = asyncHandler(async (req, res) => { const users = await User.find({}).select("-password"); // Exclude password field if (users) { res.json(users); } else { res.status(404); throw new Error("No users found"); } }); // @desc Update user by Admin // @route PUT /api/user/update-admin/:id // @access Private/Admin

const updateUserByAdmin = asyncHandler(async (req, res) => { const { id } = req.params; const { username, email, isAdmin, imageUrl } = req.body; // Find the user by id const user = await User.findById(id); if (!user) { return res.status(404).json({ success: false, message: 'User not found' }); } // Update user fields user.username = username || user.username; user.email = email || user.email; user.isAdmin = isAdmin !== undefined ? isAdmin : user.isAdmin; user.imageUrl = imageUrl || user.imageUrl; const updatedUser = await user.save(); res.json({ success: true, message: 'User updated successfully', updatedUser, }); });

const deleteUserByAdmin = async (req, res) => { try { const { userId } = req.params; const user = await User.findById(userId); if (!user) { return res.status(404).json({ message: 'User not found' }); } // Set isDeleted to true user.isDeleted = true; user.userStatus='Deactivated'; await user.save(); res.status(200).json({ message: 'User marked as deleted successfully', user }); } catch (error) { res.status(500).json({ message: error.message }); } };

const restoreUserByAdmin = async (req, res) => { try { const { userId } = req.params; const user = await User.findById(userId); if (!user) { return res.status(404).json({ message: 'User not found' }); } // Set isDeleted to false user.isDeleted = false; user.userStatus='Active'; await user.save(); res.status(200).json({ message: 'User restored successfully', user }); } catch (error) { res.status(500).json({ message: error.message }); } }; module.exports = { registerUser, loginUser, getUserProfile, updateUserProfile, requestPasswordReset, resetPassword,getAllUsers,updateUserByAdmin,deleteUserByAdmin,restoreUserByAdmin, };

< path="backend/middleware/authMiddleware.js">

const jwt = require("jsonwebtoken");

const asyncHandler = require("express-async-handler");

const User = require("../models/userModel"); // Middleware to check if the user is authenticated

const protect = asyncHandler(async (req, res, next) => { let token; if ( req.headers.authorization && req.headers.authorization.startsWith("Bearer") ) { try { // Get token from header token = req.headers.authorization.split(" ")[1]; // Verify token const decoded = jwt.verify(token, process.env.JWT_SECRET); // Get user from the token req.user = await User.findById(decoded.id).select("-password"); if (req.user && req.user.isDeleted) { return res.status(403).json({ message: "Your account has been deleted." }); } next(); } catch (error) { console.error(error); return res.status(401).json({ message: "Not authorized, no token" }); } } if (!token) { res.status(401); throw new Error("Not authorized, no token"); } });

const isAdmin = asyncHandler(async (req, res, next) => { // Ensure the user is authenticated let token; if ( req.headers.authorization && req.headers.authorization.startsWith("Bearer") ) { try { // Get token from header token = req.headers.authorization.split(" ")[1]; // Verify token const decoded = jwt.verify(token, process.env.JWT_SECRET); // Get user from the token req.user = await User.findById(decoded.id).select("-password"); if (req.user && req.user.isDeleted) { return res.status(403).json({ message: "Your account has been deleted." }); } // Check if the user is an admin if (req.user && req.user.isAdmin) { next(); } else { res.status(403); throw new Error("Not authorized as an admin"); } } catch (error) { console.error(error); res.status(401); throw new Error("Not authorized, token failed"); } } if (!token) { res.status(401); throw new Error("Not authorized, no token"); } }); module.exports = { protect, isAdmin };

< path="backend/models/cartModel.js">

const mongoose = require('mongoose');

const cartSchema = mongoose.Schema({ userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: false }, sessionId: { type: String, required: false }, items: [ { productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true }, productName: { type: String, required: true }, productDescription:{type:String}, image: { type: String, required: true }, price: { type: Number, required: true }, quantity: { type: Number, required: true, default: 1 }, totalPrice: { type: Number, required: true } } ], totalPrice: { type: Number, required: true, default: 0 }, totalQuantity: { type: Number, default: 0 }, taxAmount: { type: Number, default: 0 }, couponCode: { type: String }, discountAmount: { type: Number, default: 0 }, expiresAt: { type: Date }, isExpired: { type: Boolean, default: false } }, { timestamps: true, }); module.exports = mongoose.model('Cart', cartSchema);

< path="backend/models/OrdersDetailModel.js">

const mongoose = require('mongoose');

const OrderSchema = new mongoose.Schema( { userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }, // Detailed Order Summary order_summary: { items: [ { productId: { type: String, required: true }, productImageUrl: { type: String }, productName: { type: String, required: true }, quantity: { type: Number, required: true }, price: { type: Number, required: true }, productDescription: { type: String }, }, ], totalPrice: { type: Number, required: true }, order_date: { type: Date, default: Date.now

}, totalPrice: { type: Number, required: true }, payment_status: { type: String, enum: ['Pending', 'Paid', 'Failed', 'Refunded'], default: 'Paid' }, // Payment History for complex payment handling payment_history: [ { payment_method: { type: String }, // e.g., Credit Card, PayPal transaction_id: { type: String }, payment_status: { type: String, enum: ['Pending', 'Paid', 'Failed'], default: 'Pending' }, payment_date: { type: Date, default: Date.now } } ] }, // Billing Information billing_address: { address_line_1: { type: String}, address_line_2: { type: String }, admin_area_1: { type: String}, admin_area_2: { type: String }, postal_code: { type: String }, country_code: { type: String }, // phone: { type: String, required: true } // Optionally add phone number to billing }, email_address:{ type: String,default:"-"}, national_number: { type: String,default:"-"} , paymentSource: { type: String }, paymentID: { type: String },payment_status: { type: String, enum: ['Pending', 'COMPLETED', 'Failed'], default: 'Pending' }, // Shipping Information // Order History for tracking status changes over time order_history: [ { updated_at: { type: Date, default: Date.now }, remarks: { type: String,default:'' } // Optional remarks explaining status change } ], timestamps: { created_at: { type: Date, default: Date.now }, updated_at: { type: Date, default: Date.now } } }, { timestamps: true } // Automatically creates 'createdAt' and 'updatedAt' fields ); // Middleware to automatically update 'updated_at' field on save OrderSchema.pre('save', function (next) { this.updated_at = Date.now(); next(); });

const Order = mongoose.model('Order', OrderSchema); module.exports = Order;

< path="backend/routes/cartRoutes.js">

const express = require('express'); const { addToCart, removeFromCart, getCart, clearCart,addToCart1, removeFromCart1, getCart1, clearCart1 }= require('../controllers/cartController');

const { protect } = require('../middleware/authMiddleware'); // Ensure correct import

const router = express.Router(); // Route to get cart details (logged-in users only) router.get('/', protect,getCart); // Route to add an item to the cart (logged-in users only) router.post('/add',protect, addToCart); // Route to remove an item from the cart (logged-in users only) router.post('/remove',protect, removeFromCart); // Route to clear the cart (logged-in users only) router.delete('/clear',protect, clearCart); // Route to get cart details (logged-in users only) router.get('/food', getCart1); // Route to add an item to the cart (logged-in users only) router.post('/add1',protect, addToCart1); // Route to remove an item from the cart (logged-in users only) router.post('/remove1', protect, removeFromCart1); // Route to clear the cart (logged-in users only) router.delete('/clear1', protect, clearCart1); module.exports = router;

< path="backend/routes/foodRoutes.js">

const express = require("express");

const { isAdmin, protect } = require('../middleware/authMiddleware.js');

const router = express.Router(); const { getProducts, createProducts, getProductsById, updateProductsById, deleteProductsById, } = require("../controllers/foodsController"); router.get("/all", getProducts); router.get("/all1", isAdmin,getProducts);

router.post("/",isAdmin, createProducts); // Route for getting, updating, and deleting a product by ID router.get("/:id", getProductsById); router.put("/edit",isAdmin, updateProductsById); router.delete("/del",isAdmin, deleteProductsById); module.exports = router;

< path="backend/routes/moviesRoutes.js">

const express = require("express");

const { isAdmin, protect } = require('../middleware/authMiddleware.js');

const router = express.Router(); const { getProducts, createProducts, getProductsById, updateProductsById, deleteProductsById, } = require("../controllers/moviesController"); router.get("/all", getProducts); router.get("/all1",isAdmin, getProducts); router.post("/",isAdmin, createProducts); // Route for getting, updating, and deleting a product by ID router.get("/:id", getProductsById); router.put("/edit",isAdmin, updateProductsById); router.delete("/del",isAdmin, deleteProductsById); module.exports = router;

< path="backend/routes/OrderDetailRoutes.js">

const express = require('express');

const router = express.Router();

const orderController = require('../controllers/OrdersDetailsControllers.js');

const { isAdmin, protect } = require('../middleware/authMiddleware.js'); // Route to create a new order router.post('/orders',protect, orderController.createOrder); // Route to get all orders (Admin only) router.get('/admin/orders', isAdmin, orderController.getAllOrders); // Route to get a single order by ID for a user router.get('/my-orders/:orderId', protect, orderController.getOrderById); // Route to update order status (Admin only) router.patch('/admin/order-status/:orderId', isAdmin, orderController.updateOrderStatus); // Route to cancel an order (User only) router.patch('/my-orders/cancel/:orderId', protect, orderController.cancelOrder); // Route to add payment information (User only) router.patch('/my-orders/payment/:orderId', protect, orderController.addPaymentInfo); // Route to update shipping details (Admin only) router.patch('/admin/order-shipping/:orderId', isAdmin, orderController.updateShippingDetails); // Route to process refund (Admin only) router.patch('/admin/refund-order/:orderId', isAdmin, orderController.processRefund); // Route to get all orders (user ) router.get('/my-orders',protect, orderController.getOrders); module.exports = router;

< path="backend/routes/userRoutes.js"> // routes/userRoutes.js

const express = require("express");

const router = express.Router(); const { registerUser, loginUser, getUserProfile, updateUserProfile, resetPassword,

requestPasswordReset,getAllUsers,updateUserByAdmin,deleteUserByAdmin,restoreUserByAdmin, } = require("../controllers/userController");

const { protect, isAdmin } = require("../middleware/authMiddleware"); // Route for user registration router.post("/register", registerUser); // Route for user login router.post("/login", loginUser); // Route for getting the user profile router.get("/profile", protect, getUserProfile); // Route for updating the user profile router.put("/profile", protect, updateUserProfile); router.delete('/delete-user/:userId', isAdmin, deleteUserByAdmin); router.get("/all-users",isAdmin, getAllUsers); // Route to get all users by only admin router.put("/update-admin/:id", isAdmin, updateUserByAdmin); router.put('/restore-user/:userId', isAdmin, restoreUserByAdmin); // Route for requesting password reset router.post('/forgot-password', requestPasswordReset); // Route for resetting password router.patch('/reset-password/:resetToken', resetPassword); module.exports = router;

< path="backend/server.js">

const express = require("express");

const connectDb = require("./config/dbConnection");

const errorHandler = require("./middleware/errorHandler");

const dotenv = require("dotenv").config(); const multer=require("multer");

const cloudinary = require('cloudinary').v2;

const cors = require("cors");

const path = require("path"); connectDb();

const app = express();

const port = process.env.PORT || 5000; app.use(express.json()); app.use(cors()); app.get('/',(req,res)=>{ res.status(200).send("Express App is Running cool") }) // const storage=multer.diskStorage({ // destination:'./Upload/images', // filename:(req,file,cb)=>{ // return cb(null,${file.fieldname}_${Date.now()}${path.extname(file.originalname)}) // } // }) // const upload=multer({storage:storage}) // app.use('/images',express.static('Upload/images')) // app.use('/upload',upload.single('product'),require('./routes/imageupload'))

const uploadRoutes = require('./routes/imageupload'); // Cloudinary configuration cloudinary.config({ cloud_name: process.env.CLOUDINARY_CLOUD_NAME, api_key: process.env.CLOUDINARY_API_KEY, api_secret: process.env.CLOUDINARY_API_SECRET, }); // ROUTES BELOW app.use('/upload', uploadRoutes); // Route for Products app.use("/api/moviesproducts", require("./routes/moviesRoutes")); app.use("/api/foodproducts", require("./routes/foodRoutes")); // Route for User Registration and Authentication app.use("/api/user", require("./routes/userRoutes")); //Route for Cart app.use("/api/cart", require("./routes/cartRoutes")); //Route for

OrderDetails app.use('/api', require('./routes/OrderDetailRoutes')); app.use('/', require('./routes/commentRoutes')) app.use('/api/rating', require('./routes/ratingRoutes')); app.use('/api', require('./routes/contactRoutes')); // Error handling middleware app.use(errorHandler); // APP CONFIG START app.listen(port, () => { console.log(`Server running on port http://localhost:${port}`); });

< path="admin/src/App.jsx"> import React ,{ useState } from 'react' import Home1 from './Home/Home1'; import Home from './Home/Home'; import Signup from './Signup/Signup'; import App1 from './App1'; import PasswordReset from './PasswordReset/PasswordReset'; import { Routes, Route,Navigate } from 'react-router-dom'; import AddFoodProduct from './Components/AddProduct/AddFoodProduct'; import AddMovieProduct from './Components/AddProduct/AddMovieProduct'; import MovieProduct from './Components/MovieProduct/MovieProduct'; import FoodProduct from './Components/FoodProduct/FoodProduct'; import AllOrder from './Components/AllOrder/AllOrder'; import ProtectedRoute from './Pages/ProtectedRoute'; import ErrorPage from './Pages/ErrorPage'; import MyAccount from './Pages/MyAccount'; import AllUser from './Components/AllUser/AllUser'; import ProtectedRoute1 from './Pages/ProtectedRoute1'; function App() { return (

```
   <Routes>
      <Route path='/' element={<Home1/>}/>
      <Route path='/home' element={<ProtectedRoute element={Home}/>}/>
      <Route path='/admin/' element={<ProtectedRoute element={App1}/>}>
        <Route path='addfoodproduct' element={<AddFoodProduct/>}/>
        <Route path='addmovieproduct' element={<AddMovieProduct/>}/>
        <Route path='movieproduct' element={<MovieProduct/>}/>
        <Route path='foodproduct' element={<FoodProduct/>}/>
        <Route path='allorders' element={<AllOrder/>}/>
        <Route path='allusers' element={<AllUser/>}/>
      </Route>
      <Route path='/login' element={<ProtectedRoute1 element={Signup}/>}/>
      <Route path="error" element={<ErrorPage />} />
      <Route
      path="/account"
      element={<ProtectedRoute element={MyAccount} />}
      />
      <Route path="*" element={<Navigate to="/error" />} />
      <Route path="/reset-password/:resetToken" element={<ProtectedRoute1
element={PasswordReset}/>} />
    </Routes>
</div>
```

) } export default App

< path="admin/src/Components/AddProduct/AddFoodProduct.jsx"> import React, { useState } from 'react' import './AddProduct.css' import upload_area from '../../assets/upload_area.svg'

const AddFoodProduct = () => { const [image,setImage]=useState(false); const [productDetails,setProductDetails]=useState({ productName:“”, productPrice:””, available:true, }); const imageHandler=(e)=>{ setImage(e.target.files[0]); } const changeHandler=(e)=>{ setProductDetails({…productDetails,[e.target.name]:e.target.value}) } const Add_Product=async()=>{ console.log(productDetails); let responseData; let product=productDetails; let formData=new FormData(); formData.append(‘product’,image); const token = localStorage.getItem(‘auth-token’); // Retrieve the token if (!token) { console.error(‘No token found in localStorage’); return; } await fetch(${import.meta.env.VITE_BACKEND_URL}/upload/image,{ method:‘POST’, headers:{ Accept:‘application/json’, }, body:formData, }).then((resp)=>resp.json()).then((data)=>{responseData=data}) if (responseData.success){ product.image=responseData.image_url; console.log(product) await fetch(${import.meta.env.VITE_BACKEND_URL}/api/foodproducts/, { method:‘POST’, headers:{ Accept:‘application/json’,”Content-Type”: “application/json”, Authorization: Bearer ${token} }, body:JSON.stringify(product), } ).then((resp)=>resp.json()).then((data)=>{ data.success?alert(“Product Added”):alert(“Failed”) setProductDetails({ productName:“”, productPrice:””, available:true, }) setImage(false); }) } } return (

```
  <h1>Add Food Items</h1>
  <div className="addproduct-itemfield">
    <p>Product Title</p>
    <input type="text" value={productDetails.productName}
onChange={changeHandler} name='productName' placeholder='Type product name
here ' />
  </div>
  <div className="addproduct-price">
  <div className="addproduct-itemfield">
    <p>Product Price</p>
    <input type="text" value={productDetails.productPrice}
onChange={changeHandler} name='productPrice' placeholder='Type product price
here ' />
  </div>
  </div>
  <div className="addproduct-itemfield">
  <p>Product Available</p>
  <select value={productDetails.available} onChange={changeHandler}
name="available" className='add-product-selector'>
  <option value="true">true</option>
  <option value="false">false</option>
  </select>
  </div>
  <div className="addproduct-itemfield">
  <label htmlFor="file-input">
    <img src={image?URL.createObjectURL(image):upload_area}
className='addproduct-thumbnail-img' alt="" />
  </label>
  <input onChange={imageHandler} type="file" name='image' id='file-input'
```

```
hidden/>
  </div>
  <button onClick={()=>{Add_Product()}} className='addproduct-btn'>Add
Product</button>
</div>
```

) } export default AddFoodProduct

< path="admin/src/Components/AddProduct/AddMovieProduct.jsx"> import React, { useState } from 'react' import './AddProduct.css' import upload_area from '../../assets/upload_area.svg'

const AddMovieProduct = () => { const [image,setImage]=useState(false); const [productDetails,setProductDetails]=useState({ productName:"", category:"", productDescription:"", productPrice:"", available:true, }); const imageHandler=(e)=>{ setImage(e.target.files[0]); } const changeHandler=(e)=>{ setProductDetails({...productDetails,[e.target.name]:e.target.value}) } const Add_Product=async()=>{ console.log(productDetails); let responseData; let product=productDetails; let formData=new FormData(); formData.append('product',image); const token = localStorage.getItem('auth-token'); // Retrieve the token if (!token) { console.error('No token found in localStorage'); return; } await fetch(${import.meta.env.VITE_BACKEND_URL}/upload/image,{ method:'POST', headers:{ Accept:'application/json', }, body:formData, }).then((resp)=>resp.json()).then((data)=>{responseData=data}) if (responseData.success){ product.image=responseData.image_url; console.log(product) await fetch(${import.meta.env.VITE_BACKEND_URL}/api/moviesproducts/, { method:'POST', headers:{ Accept:'application/json',"Content-Type": "application/json", Authorization: Bearer ${token} }, body:JSON.stringify(product), } ).then((resp)=>resp.json()).then((data)=>{ data.success?alert("Product Added"):alert("Failed") setProductDetails({ productName:"", category:"", productDescription:"", productPrice:"", available:true, }) setImage(false); }) } } return (

```
  <h1>Add Movies</h1>
  <div className="addproduct-itemfield">
    <p>Product Title</p>
    <input type="text" value={productDetails.productName}
onChange={changeHandler} name='productName' placeholder='Type product name
here ' />
  </div>
  <div className="addproduct-itemfield">
    <p>Product Description</p>
    <input type="text" value={productDetails.productDescription}
onChange={changeHandler}  name='productDescription' placeholder='Type product
desc. here ' />
  </div>
  <div className="addproduct-price">
  <div className="addproduct-itemfield">
    <p>Product Price</p>
    <input type="text" value={productDetails.productPrice}
```

```
onChange={changeHandler}  name='productPrice' placeholder='Type product price
here ' />
  </div>
  </div>
  <div className="addproduct-itemfield">
  <p>Product Category</p>
  <input type="text" value={productDetails.category} onChange={changeHandler}
name='category' placeholder='Type product genre here ' />
  </div>
  <div className="addproduct-itemfield">
  <p>Product Available</p>
  <select name="available" value={productDetails.available}
onChange={changeHandler} className='add-product-selector'>
  <option value="true">true</option>
  <option value="false">false</option>
  </select>
  </div>
  <div className="addproduct-itemfield">
  <label htmlFor="file-input">
    <img src={image?URL.createObjectURL(image):upload_area}
className='addproduct-thumbnail-img' alt="" />
  </label>
  <input onChange={imageHandler} type="file" name='image' id='file-input'
hidden/>
  </div>
  <button onClick={()=>{Add_Product()}} className='addproduct-btn'>Add
Product</button>
</div>
```

) } export default AddMovieProduct

< path="admin/src/Pages/Admin/Admin.jsx"> import React from 'react' import './Admin.css' import Sidebar from '../../Components/Sidebar/Sidebar' import {Routes,Route} from 'react-router-dom' import MovieProduct from '../../Components/MovieProduct/MovieProduct' import FoodProduct from '../../Components/FoodProduct/FoodProduct' import AddFoodProduct from '../../Components/AddProduct/AddFoodProduct' import AddMovieProduct from '../../Components/AddProduct/AddMovieProduct' import AllOrder from '../../Components/AllOrder/AllOrder'; import AllUser from '../../Components/AllUser/AllUser' function Footer1() { return (

```
    <a href="https://www.facebook.com/"><i className="fab fa-facebook-
f"></i></a>
    <a href="https://twitter.com/LOGIN"><i className="fab fa-
twitter"></i></a>
    <a href="https://www.instagram.com/"><i className="fab fa-
instagram"></i></a>
    <a href="https://in.pinterest.com/"><i className="fab fa-
pinterest"></i></a>
  </div>
```

```
  <span style={{ color: "white" }}>ReelBite: Cinematic Cravings Admin's
Page</span>
</footer>
```

); }

const Admin = () => { return (

```
  <Sidebar/>
  <Routes>
  <Route path='addfoodproduct' element={<AddFoodProduct/>}/>
    <Route path='addmovieproduct' element={<AddMovieProduct/>}/>
    <Route path='movieproduct' element={<MovieProduct/>}/>
    <Route path='foodproduct' element={<FoodProduct/>}/>
    <Route path='allorders' element={<AllOrder/>}/>
    <Route path='allusers' element={<AllUser/>}/>
  </Routes>
  <Footer1 />
</div>
```

) } export default Admin

< path="frontend/src/AddtoCart/AddtoCart.jsx"> import React, { useState,useEffect,useContext } from 'react' import Navbar from '../Component/Navbar/Navbar3' import './AddtoCart.css'; import blankCart from "../assets/emptyCart.svg"; import { movieitems ,fooditems } from '../Data'; import CartItem from './CartItem'; import { MovieContext } from '../Context/MovieContext'; import './Exit34.css'; import movie from '../Food/pics6/make.jpg' import CartFooter from './CartFooter'; function AddtoCart() { function Footer13431() { return (

```
      <a href="https://www.facebook.com/"><i className="fab fa-facebook-
f"></i></a>
      <a href="https://twitter.com/LOGIN"><i className="fab fa-
twitter"></i></a>
      <a href="https://www.instagram.com/"><i className="fab fa-
instagram"></i></a>
      <a href="https://in.pinterest.com/"><i className="fab fa-
pinterest"></i></a>
    </div>
    <span style={{ color: "white" }}>ReelBite: Cinematic Cravings Food
Page</span>
  </footer>
);
```

} const [movieCartItems, setMovieCartItems] = useState([]); const [foodCartItems, setFoodCartItems] = useState([]); const [errorMessage, setErrorMessage] = useState(''); const [total1, setTotal] = useState(0); const[quantityAdd,setQuantityAdd] = useState(false); const[cartdata,setCartdata]=useState([]); // Fetch movie cart items from the backend useEffect(() => { const fetchMovieCartItems = async () => { const token = localStorage.getItem('auth-token'); // Retrieve the token if (!token) { console.log('No token

found in localStorage'); return; } try { const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/cart/,{ method:'GET', headers: { 'Content-Type': 'application/json', Authorization: Bearer ${token} // Set the Authorization header } }); if (!response.ok) { throw new Error(HTTP error! Status: ${response.status}); } const data = await response.json(); setMovieCartItems(data.items || []); setCartdata(data); setTotal(data.totalPrice) } catch (error) { setErrorMessage('Failed to load cart items:' + error.message); } }; fetchMovieCartItems(); }, []);

const handleAddQuantity = async (productId) => { setQuantityAdd(false); const token = localStorage.getItem('auth-token'); try { const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/cart/add, { method: 'POST', headers: { 'Content-Type': 'application/json', Authorization: Bearer ${token}, }, body: JSON.stringify({ productId, quantity: 1 }), }); const updatedCart = await response.json(); setQuantityAdd(true); setMovieCartItems(updatedCart.items); setTotal(updatedCart.totalPrice); } catch (error) { setErrorMessage('Error updating cart:' + error.message); } };

const handleRemoveQuantity = async (productId) => { setQuantityAdd(false); const token = localStorage.getItem('auth-token'); try { const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/cart/remove, { method: 'POST', headers: { 'Content-Type': 'application/json', Authorization: Bearer ${token}, }, body: JSON.stringify({ productId, quantity: 1 }), }); const updatedCart = await response.json(); setMovieCartItems(updatedCart.items); setQuantityAdd(true); setTotal(updatedCart.totalPrice); } catch (error) { setErrorMessage('Error updating cart:' + error.message); } };

const handledeleteQuantity = async (productId) => { const token = localStorage.getItem('auth-token'); try { const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/cart/remove1, { method: 'POST', headers: { 'Content-Type': 'application/json', Authorization: Bearer ${token}, }, body: JSON.stringify({ productId}), }); const updatedCart = await response.json(); setMovieCartItems(updatedCart.items); setTotal(updatedCart.totalPrice); } catch (error) { setErrorMessage('Error updating cart:' + error.message); } }; const [exitIntent, setExitIntent] = useState(false); useEffect(() => { const handleMouseLeave = () => { setExitIntent(true); }; const handleMouseOut = (event) => { if (event.clientY < 0 && exitIntent) { document.getElementById('exit-popup1121').style.display = 'block'; } }; document.addEventListener('mouseleave', handleMouseLeave); window.addEventListener('mouseout', handleMouseOut); return () => { document.removeEventListener('mouseleave', handleMouseLeave); window.removeEventListener('mouseout', handleMouseOut); }; }, [exitIntent]); const handleClosePopup = () => { document.getElementById('exit-popup1121').style.display = 'none'; }; const{cartItems,cartItem,item,itema} = useContext(MovieContext) return (

```
    <Navbar quantityAdded = {quantityAdd}/>
    <div className='cartContainer' >
      <div className='CartLeft'>
        <div className='cartHeader'>
```

```
          <div className='headerLeft'>
              <h3>PRODUCT</h3>
          </div>
          <div className='headerRight'>
          <h3>PRICE</h3>
          <h3>QUANTITY</h3>
          <h3>TOTAL</h3>
          </div>
        </div>
        {movieCartItems.length > 0 ? (
                movieCartItems.map(item => (
                    <CartItem
                        key={item.productId}
                        item={item}
                        onAdd={handleAddQuantity}
                        onRemove={handleRemoveQuantity}
                        ondelete={handledeleteQuantity}
                    />
                ))
            ) : (
              <div className="blankCartContainer">
              <img src={blankCart} alt="Empty cart" />
              <div className="cartHeading">
                <h1>Your Cart is Empty!!</h1>
              </div>
            </div>
            )}
      </div>
    </div>
      <CartFooter  total={total1} cartItems={cartdata}/>
    <Footer13431/>
    <div id="exit-popup1121" style={{ display: 'none' }} >
        <h2>Don't Leave Yet!</h2>
        <h4>We have an exciting offer for you: </h4>
        <h4>Use Promocode 'Food233' to get 30% Cashback <br />on total food
price, upto Rs. 100.</h4>
        <img src={movie} alt="Offer" width="250" height="250" />
    <br />
    <button id='continue-button'onClick={handleClosePopup}>Continue</button>
  </div>
</div>
```

) } export default AddtoCart

< path="frontend/src/AddtoCart/CartItem.jsx"> import React, { useContext } from "react"; import './CartItem.css'; import { MovieContext } from "../Context/MovieContext"; import {faPlus,faMinus,faTrash} from '@fortawesome/free-solid-svg-icons'; import { FontAwesomeIcon } from '@fortawesome/react-fontawesome'

const CartItem = ({ item,onAdd, onRemove,ondelete }) => { const { productId, image, productName, productDescription, price, quantity } = item; return (

```
        <div className="itemContainer">
            <div className="itemLeft">
                <img src={image} alt={productName} />
                <span>{productName}</span>
                {productDescription !== "" ? (
                    <div>
                        <span>{productDescription}</span>
                    </div>) : null
                }
            </div>
            <div className="itemRight">
                <div className="rightContainer">
                    <span>${price}</span>
                    <span>
                        {/* <span onClick={() => onAdd(productId)}> */}
                        <span onClick={() => onAdd(productId)}>
                            <FontAwesomeIcon
                                style={{ borderRadius: "5px",
backgroundColor: "lightgray" }}
                                icon={faPlus}
                            />
                        </span>
                        {quantity}
                        {/* <span onClick={() => onRemove(productId)}> */}
                        <span onClick={() => onRemove(productId)}>
                            <FontAwesomeIcon
                                style={{ marginLeft: "5px", borderRadius:
"5px", backgroundColor: "lightgray" }}
                                icon={faMinus}
                            />
                        </span>
                        <span onClick={() => ondelete(productId)}>
                        <FontAwesomeIcon
                        style={{ marginLeft: "5px", borderRadius: "5px",
backgroundColor: "lightgray" }}
                        icon={faTrash} />
                        </span>
                    </span>
                    <span>${quantity * price}</span>
                </div>
            </div>
        </div>
    </div>
);
```

} export default CartItem;

< path="frontend/src/App.jsx"> import React, { useState, useEffect } from 'react'; import Home1 from './Home/Home1'; import Home from './Home/Home'; import Signup from './Signup/Signup'; import About from './About/About'; import Contactus from './Contantus/Contanctus'; import Login from './Login/Login'; import { Routes, Route,Navigate } from 'react-router-dom'; import Movies from './Movies/Movies'; import PasswordReset from './PasswordReset/PasswordReset'; import Food from './Food/Food'; import AddtoCart from './AddtoCart/AddtoCart'; import AllOrders from './AllOrders/AddtoCart'; import { MovieContextProvider } from './Context/MovieContext'; import { PayPalScriptProvider } from '@paypal/react-paypal-js'; import ProductDetails from './Component/ProductDetails'; import ErrorPage from './Component/ErrorPage'; import ProtectedRoute from './Component/ProtectedRoute'; import MyAccount from './Component/MyAccount'; import ProtectedRoute1 from './Component/ProtectedRoute1';

const App = () => { return (  <>  <div style={{ overflowX:'hidden' }} >  }/> }/> }/> }/> }/> }/> }/> }/> }/> }/> } /> } /> }/> } /> } /> } />

```
  </MovieContextProvider>
  </>
  </PayPalScriptProvider>
```

); }; export default App;

< path="frontend/src/Component/ProductDetails.jsx"> import React, { useEffect, useState } from 'react'; import { useNavigate, useParams } from 'react-router-dom'; import Navbar from './Navbar/Navbar'; import StarRate from '../Component/StarRating/StarRate'; import Comment from '../Comments/Comment'; import blankproduct from "../assets/nop.png"; import './ProductDetails.css'; import { ToastContainer, toast } from "react-toastify"; import { PulseLoader } from "react-spinners"; import "react-toastify/dist/ReactToastify.css"; function Footer13444() { return (

```
    <a href="https://www.facebook.com/"><i className="fab fa-facebook-
f"></i></a>
    <a href="https://twitter.com/LOGIN"><i className="fab fa-
twitter"></i></a>
    <a href="https://www.instagram.com/"><i className="fab fa-
instagram"></i></a>
    <a href="https://in.pinterest.com/"><i className="fab fa-
pinterest"></i></a>
  </div>
  <span style={{ color: "white" }}>ReelBite: Cinematic Cravings Movies
Page</span>
</footer>
```

); }

const ProductDetails = () => { const navigate = useNavigate(); const { productId } = useParams(); // Extract productId from the URL const [product, setProduct] = useState(null); const [loading, setLoading] = useState(true); const [isLoading, setIsLoading] = useState(false); const [quantityAdd, setQuantityAdd] = useState(false); const

[loadingStates, setLoadingStates] = useState({}); const handleClick = (product) => { const token = localStorage.getItem("auth-token"); if (!token) { toast.error("Login/Signup first"); // Show the alert return; } setLoadingStates((prevStates) => ({ ...prevStates, [product.products_id]: true, })); setTimeout(() => { handleAddToCart(product._id); setLoadingStates((prevStates) => ({ ...prevStates, [product.products_id]: false, })); }, 1000); }; const handleAddToCart = async (productId, quantity = 1) => { setQuantityAdd(false); const t = localStorage.getItem('auth-token'); if (!t) { toast.error("Login/Signup first"); // Show the alert return; } try { const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/cart/add, { method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': Bearer ${localStorage.getItem('auth-token')} // Add auth token if user is logged in }, body: JSON.stringify({ productId, quantity }) }); if (!response.ok) { toast.error('Failed to add item to cart'); throw new Error('Failed to add item to cart'); } setQuantityAdd(true); toast.success('Item added to cart successfully!'); } catch (error) { console.error('Error adding product to cart:', error); toast.error('Error adding product to cart'); } }; useEffect(() => { // Function to fetch product details from multiple endpoints const fetchProductDetails = async () => { const endpoints = [ ${import.meta.env.VITE_BACKEND_URL}/api/moviesproducts/${productId}, ${import.meta.env.VITE_BACKEND_URL}/api/foodproducts/${productId} ]; for (const endpoint of endpoints) { try { const response = await fetch(endpoint); if (response.ok) { const data = await response.json(); setProduct(data); // Set product if found setLoading(false); return; } } catch (error) { console.error(Error fetching from ${endpoint}:, error); } } // If no product was found in any of the endpoints setLoading(false); }; fetchProductDetails(); }, [productId]); // Handle loading and blank state within the render logic return ( <> {loading ? (

Loading product details...

```
  ) : !product ? (
    <div className="blankCartContainer21">
      <img src={blankproduct} alt="Empty cart" />
    </div>
  ) : (
     <div className="product-details">
       <div className="product-image">
         <img src={product.image} alt={product.productName} />
       </div>
       <div className="product-info">
         <h2>{product.productName}</h2>
         <p>Price: Rs. {product.productPrice}</p>
         {product.productDescription && (
           <p>Description: {product.productDescription}</p>
         )}
         <p>Category: {product.category ? product.category : "Food"}</p>
         <p>Available: {product.available ? "Yes" : "No"}</p>
         <div className="productPriceContainer">
           <StarRate userId={localStorage.getItem("id")}
productId={product._id} productModel="allProducts"/>
```

```
        </div>
        {product.available ? ( // Render button only if product is available
          <div className="productAddToCartButton">
            <button
              className={loadingStates[product._id] ? "button loading" :
"button"}
              onClick={() => handleClick(product)}
              disabled={loadingStates[product._id]}
            >
              <span>Add to cart</span>
              <div className="cart">
                <svg viewBox="0 0 36 26">
                  <polyline points="1 2.5 6 2.5 10 18.5 25.5 18.5 28.5 7.5
7.5 7.5"></polyline>
                  <polyline points="15 13.5 17 15.5 22 10.5"></polyline>
                </svg>
              </div>
            </button>
          </div>
        ) : (
          <p style={{ color: 'red' }}>This product is currently
unavailable.</p>
        )}
      </div>
      <div className='pc'>
        Reviews <br />
        <br />
        <Comment productId={product._id} />
      </div>
    </div>
  )}
  <Footer13444 />
  <ToastContainer />
</>
```

); }; export default ProductDetails;

< path="frontend/src/Food/Food.jsx"> import React, { useState, useEffect,useContext } from 'react'; import { Link, useNavigate } from 'react-router-dom'; import './Food.css' import blankproduct from "../assets/nop.png"; import Navbar1 from '../Component/Navbar1/Navbar1'; import movie from './pics6/make.jpg' import './Exit49.css' import { fooditems } from '../Data'; import Comment from '../Comments/Comment' import StarRate from '../Component/StarRating/StarRate'; import { MovieContext } from '../Context/MovieContext'; import { ToastContainer, toast } from "react-toastify"; import { PulseLoader } from "react-spinners"; import "react-toastify/dist/ReactToastify.css"; function Footer1343() { return (

```
      <a href="https://www.facebook.com/"><i className="fab fa-facebook-
f"></i></a>
      <a href="https://twitter.com/LOGIN"><i className="fab fa-
```

```
twitter"></i></a>
      <a href="https://www.instagram.com/"><i className="fab fa-
instagram"></i></a>
      <a href="https://in.pinterest.com/"><i className="fab fa-
pinterest"></i></a>
    </div>
    <span style={{ color: "white" }}>ReelBite: Cinematic Cravings Food
Page</span>
  </footer>
);

}
```

const Food = ()=>{ // const {addToFoodCart, CartItems} = useContext(MovieContext) const [movieItems, setMovieItems] = useState([]); const [exitIntent, setExitIntent] = useState(false); const [searchTerm, setSearchTerm] = useState(''); const [isLoading, setIsLoading] = useState(false); const[quantityAdd,setQuantityAdd] = useState(false); const navigate=useNavigate(); const handleProductClick = (productId) => { navigate(`/product/${productId}`); // Use navigate to go to the product details page }; useEffect(() => { const fetchMovies = async () => { try { console.log('Backend URL:', import.meta.env.VITE_BACKEND_URL); const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/api/foodproducts/all`); const data = await response.json(); setMovieItems(data); // Store fetched movie data in state } catch (error) { console.error('Error fetching food items:', error); } }; fetchMovies(); const handleMouseLeave = () => { setExitIntent(true); }; const handleMouseOut = (event) => { if (event.clientY < 0 && exitIntent) { document.getElementById('exit-popup111').style.display = 'block'; } }; document.addEventListener('mouseleave', handleMouseLeave); window.addEventListener('mouseout', handleMouseOut); return () => { document.removeEventListener('mouseleave', handleMouseLeave); window.removeEventListener('mouseout', handleMouseOut); }; }, [exitIntent]); const handleClosePopup = () => { document.getElementById('exit-popup111').style.display = 'none'; }; const filteredMovies = movieItems.filter((movie) => movie.productName.toLowerCase().includes(searchTerm) ); const addToCart = async (productId, quantity = 1) => { setQuantityAdd(false); const t=localStorage.getItem('auth-token'); if(!t){ toast.error('Failed to add Food to cart:Login/Signup first'); return; } try { const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/api/cart/add`, { method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${localStorage.getItem('auth-token')}` // Add auth token if user is logged in }, body: JSON.stringify({ productId, quantity }) }); if (!response.ok) { toast.error('Failed to add item to cart'); throw new Error('Failed to add item to cart'); } const data = await response.json(); setQuantityAdd(true); toast.success('Food item added to cart successfully!'); } catch (error) { console.error('Error adding product to cart:', error); toast.error('Error adding product to cart'); } }; return( <>

```
        <Navbar1 setSearchTerm={setSearchTerm} quantityAdded =
{quantityAdd}/>
        <div className='movieContainer'>
      <div className='movieitemdiv'>
```

```
      {filteredMovies.length > 0 ? (
        filteredMovies.map((data, index) => (
          <div key={index} className='movieCardCont'>
            <img className='movieCardImg' src={data.image} alt={data.name}
onClick={() => handleProductClick(data._id)} // Navigate to the product
details page on click
                style={{ cursor: "pointer" }} />
            <div className='movieCardRate'  onClick={() =>
handleProductClick(data._id)} // Navigate to the product details page on
click
                style={{ cursor: "pointer" }} >
              <span>{data.productName}</span>
              </div>
            <div className='movieCardRate' >
              <StarRate userId={localStorage.getItem("id")}
productId={data._id} productModel="foods" />
            </div>
            {/* <span>{data.category}</span><br /> */}
            <span>${data.productPrice}</span>
            <span>Available: {data.available ? "Yes" : "No"}     </span>
            {data.available ? ( // Render button only if product is available
          <button className='com' onClick={() => addToCart(data._id)}>Buy
Food</button>
        ) : (
          <p style={{ color: 'red' }}>This product is currently
unavailable.</p>
        )}
            {/* <Comment productId={data._id}/> */}
          </div>
        ))
      ) : (
        <div className="blankCartContainer2">
              <img src={blankproduct} alt="Empty cart" />
              <div className="cartHeading2">
              </div>
            </div>)}
      </div>
    </div>
        <Footer1343/>
        <div id="exit-popup111" style={{ display: 'none' }} >
        <h2>Don't Leave Yet!</h2>
        <h4>We have an exciting offer for you: </h4>
        <h4>Use Promocode 'Food233' to get 30% Cashback <br />on total food
price, upto Rs. 100.</h4>
        <img src={movie} alt="Offer" width="250" height="250" />
    <br />
    <button id='continue-button'onClick={handleClosePopup}>Continue</button>
  </div>
    </div>
    <ToastContainer />
```

```
    </>
)
```

} export default Food

< path="frontend/src/Home/Home.jsx"> import React, { useState, useEffect } from 'react'; import { Link } from 'react-router-dom'; import './Home.css'; import './Exit.css'; import kung from './pics/kung.avif'; import make from './pics/make.jpg'; import love from './pics/download.jpg'; import garfiel from './pics/garfiel.avif'; import reas from './pics/13.jpg'; import food from './pics/OIP (1).jpg'; import food1 from './pics/OIP.jpg'; import pp1 from './pics/pp1.png'; import image91 from './pics/image91.jpg'; import movie from './pics/movie.mp4'; function Header() { return (

```
    <div className="container">
      <Link to='/home'><img src={pp1} alt="logoimg" width="50" height="50"
/></Link>
    </div>
    <ul className="nav-list">
      <li><Link className="bollywood" to='/movies' style={{ textDecoration:
"None" }}>Movies</Link></li>
      <li><Link className="Food" to='/food' style={{ textDecoration: "None"
}}>Food</Link></li>
      <li><Link className="about"  style={{ textDecoration: "None" }}
to='/about'  >About</Link> </li>
      <li><Link className="Contactus"  style={{ textDecoration: "none" }}
to='/contact'>Contact Us</Link></li>
      <li><Link className="bollywood" to='/account' style={{ textDecoration:
"None" }}>MyAccount</Link></li>
      {localStorage.getItem('auth-token')?<button  id="btn1"
onClick={()=>{localStorage.removeItem('auth-
token');localStorage.removeItem('id');localStorage.removeItem('avatar');windo
w.location.replace('/')}}>Logout</button>
      :<Link  style={{ textDecoration: "None" }} id="btn1" className="btn
btn-full" to='/login'>Log In</Link>}
      {/* <li><Link  style={{ textDecoration: "None" }} id="btn1"
className="btn btn-full" to='/login'>Log In</Link></li> */}
    </ul>
  </div>
  <div className="main-content-header1">
    <h1>DO YOU WANT TO <br /><span className="color">EXPLORE SOMETHING
?</span><br /></h1>
    {/* <Link  id="btn1" className="btn btn-down" to='/sign'> Sign up</Link>
*/}
  </div>
</header>
```

); } function MainSection1() { return (

```
    <div className="title">
      <h2>Today's News</h2>
```

```
    </div>
    <div className="design-content">
      <div className="design-item">
        <div className="design-img">
          <img src={reas} alt="latastmov" />
        </div>
        <div className="design-title"></div>
      </div>
      <div className="design-item">
        <p><h2></h2></p>
        <p><h1><b>"Blockbuster Movie"</b></h1></p>
        <p><h2><b>13 Reasons Why:</b></h2><i>It is an American teen drama
television series developed for Netflix by Brian Yorkey and based on the 2007
novel Thirteen Reasons Why by author Jay Asher. The series revolves around
high school student Clay Jensen and the aftermath of the suicide of fellow
student Hannah Baker. Before her death, she leaves behind a box of cassette
tapes in which she details the reasons why she chose to kill herself as well
as the people.</i></p>
      </div>
    </div>
  </div>
</section>
```

); } function BlogSection2() { return (

```
    <div className="title">
      <h2> Upcoming Movies:)</h2>
    </div>
    <div className="blog-content">
      <div className="blog-item">
        <div className="blog-img">
          <img src={kung} alt="mov1" />
          <span><i className="far fa-heart"></i></span>
        </div>
        <div className="blog-text">
          <br />
          <span>15 Mar, 2024</span>
          <h2 className="hey">Kung Fu Panda 4</h2>
          <p><b>The Unplanned Place</b></p>
          <p>Read More..</p>
        </div>
      </div>
      <div className="blog-item">
        <div className="blog-img">
          <img src={love} alt="mov2" />
          <span><i className="far fa-heart"></i></span>
        </div>
        <div className="blog-text">
          <span>22 March, 2024</span>
          <h2 className="hey">Love Lies Bleeding</h2>
```

```
          <p><b>The Unplanned Place</b></p>
          <p>Read More..</p>
        </div>
      </div>
      <div className="blog-item">
        <div className="blog-img">
          <img src={garfiel} alt="mov3" />
          <span><i className="far fa-heart"></i></span>
        </div>
        <div className="blog-text">
          <span>24 May, 2024</span>
          <h2 className="hey">Garfield</h2>
          <p><b>The Unplanned Place</b></p>
          <p>Read More..</p>
        </div>
      </div>
    </div>
  </div>
</section>
```

); } function BlogSection1() { return (

```
    <div className="title">
      <h2>Best Selling Deals:)</h2>
    </div>
    <div className="blog-content">
      <div className="blog-item">
        <div className="blog-img">
          <img src={food1} alt="food1" />
          <span><i className="far fa-heart"></i></span>
        </div>
        <div className="blog-text">
          <br />
          <span>30 April, 2021</span>
          <h2 className="hey">Pizza Love</h2>
          <p><b>One Mile At A Time</b></p>
          <p>Read More..</p>
        </div>
      </div>
      <div className="blog-item">
        <div className="blog-img">
          <img src={food} alt="food2" />
          <span><i className="far fa-heart"></i></span>
        </div>
        <div className="blog-text">
          <span>6 March, 2021</span>
          <h2 className="hey">Wrap Love</h2>
          <p><b>The Unplanned Place</b></p>
          <p>Read More..</p>
        </div>
```

```
      </div>
      <div className="blog-item">
        <div className="blog-img">
          <img src={make} alt="food3" />
          <span><i className="far fa-heart"></i></span>
        </div>
        <div className="blog-text">
          <span>2 May, 2021</span>
          <h2 className="hey">Combo Love</h2>
          <p><b>The Unplanned Place</b></p>
          <p>Read More..</p>
        </div>
      </div>
    </div>
  </div>
</section>
```

); } function Footer1() { return (

```
    <a href="https://www.facebook.com/"><i className="fab fa-facebook-
f"></i></a>
    <a href="https://twitter.com/LOGIN"><i className="fab fa-
twitter"></i></a>
    <a href="https://www.instagram.com/"><i className="fab fa-
instagram"></i></a>
    <a href="https://in.pinterest.com/"><i className="fab fa-
pinterest"></i></a>
  </div>
  <span style={{ color: "white" }}>ReelBite: Cinematic Cravings Home
Page</span>
</footer>
```

); }

const Home = () => { const [exitIntent, setExitIntent] = useState(false); useEffect(() => { const handleMouseLeave = () => { setExitIntent(true); }; const handleMouseOut = (event) => { if (event.clientY < 0 && exitIntent) { document.getElementById('exit-popup').style.display = 'block'; } }; document.addEventListener('mouseleave', handleMouseLeave); window.addEventListener('mouseout', handleMouseOut); return () => { document.removeEventListener('mouseleave', handleMouseLeave); window.removeEventListener('mouseout', handleMouseOut); }; }, [exitIntent]); const handleClosePopup = () => { document.getElementById('exit-popup').style.display = 'none'; }; return (

```
  <Header/>
  <MainSection1/>
  <BlogSection2/>
  <BlogSection1/>
  <Footer1 />
  <div id="exit-popup" style={{ display: 'none' }} >
```

```
    <h2>Tusi Jare ho??</h2>
    <h2>Tusi Na Jao Naaa!!!</h2>
    <h4>Get Exciting Offers Ahead!!!</h4>
    <h2>Watch Trailer</h2>
    <video width="300" height="300" controls>
      <source src={movie} type="video/mp4" />
      Your browser does not support the video tag.
    </video>
    <br />
    <button id='continue-button'onClick={handleClosePopup}>Continue</button>
  </div>
</div>
```

); }; export default Home;

< path="frontend/src/Login/Login.jsx"> import React, { useState,useEffect } from 'react'; import './Login.css'; import { Link, useNavigate } from 'react-router-dom'; import eyeIcon from "../assets/eye.png"; import eyeSlashIcon from "../assets/eye-2.png"; import { ToastContainer, toast } from "react-toastify"; import { PulseLoader } from "react-spinners"; import "react-toastify/dist/ReactToastify.css"; import Cookies from "js-cookie";

const Login = () => { const navigate = useNavigate(); const [email, setEmail] = useState(""); const [isLoginFormVisible, setLoginFormVisible] = useState(true); const [isLoading, setIsLoading] = useState(false); const [isLostPasswordFormVisible, setLostPasswordFormVisible] = useState(false); const [formData, setFormData] = useState({ email: email, password:"" }); const [showPassword, setShowPassword] = useState(false); const [resetEmail, setResetEmail] = useState(""); const togglePasswordVisibility = () => { setShowPassword((prev) => !prev); // Toggle the password visibility }; const changeHandle = (e) => { const { name, value } = e.target; if (name === 'email') { setEmail(value); // Only update the email state if the input name is"email" } setFormData({ ...formData, [e.target.name]: e.target.value }); }; const [rememberMe, setRememberMe] = useState(false); const handleResetEmailChange = (e) => { setResetEmail(e.target.value); }; useEffect(() => { // Check if the user email is stored in cookies when the component mounts const savedEmail = Cookies.get("email"); // const savedpass = Cookies.get("password"); if (savedEmail ) { setEmail(savedEmail); // Auto-fill the email field setFormData((prevFormData) => ({ ...prevFormData, email: savedEmail, // Correctly update the 'email' field in formData })) setRememberMe(true); // Set Remember Me to true } }, []); const login = async (event) => { event.preventDefault(); // Prevent the default form submission behavior let responseData; try { setIsLoading(true); const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/user/login, { method: 'POST', headers: { Accept: 'application/json', 'Content-Type': 'application/json' }, body: JSON.stringify(formData), }); responseData = await response.json(); setIsLoading(false); if (responseData.token) { localStorage.setItem('auth-token', responseData.token); localStorage.setItem('id', responseData._id); localStorage.setItem('avatar', responseData.imageUrl|| "https://res.cloudinary.com/dwprhpk9r/image/upload/v1728546051/uploads/product_1728546048771.png.png"); if (rememberMe) { Cookies.set("email", email, { expires: 7 }); // Store the email in cookies for 7 days // Cookies.set("password", password, { expires: 7

}); } else { Cookies.remove("email"); // Remove the email from cookies if not remembering // Cookies.remove("password"); } console.log("Login successful"); toast.success("You are logged in… WELCOME TO … !!"); setTimeout(() => { if (responseData.role === 'admin') { console.log("Admin login successful"); navigate('/home'); } else { console.log("User login successful"); navigate('/home'); } }, 2000); } else { console.log(responseData.error); toast.error("Login failed, please try again." + responseData.error); } } catch (error) { console.error("Error during login:", error); toast.error("An error occurred during login."); } }; const requestPasswordReset = async (event) => { event.preventDefault(); try { setIsLoading(true); const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/user/forgot-password, { method: 'POST', headers: { Accept: 'application/json', 'Content-Type': 'application/json' }, body: JSON.stringify({ email: resetEmail }), }); const responseData = await response.json(); setIsLoading(false); if (responseData.success) { toast.success("Password reset link sent!"); setTimeout(() => { navigate('/sign'); setLostPasswordFormVisible(false); }, 2000); } else { setIsLoading(false); toast.error("Failed to send password reset link." + responseData.message); } } catch (error) { setIsLoading(false); toast.error("An error occurred while requesting password reset."); } }; const toggleForm = (formType) => { setLoginFormVisible(formType === 'login'); setLostPasswordFormVisible(formType === 'lostPassword'); }; return (

```
  <div className="login-page1">
  <div className="box1">
  <div className="form1">
    <h1>Login</h1>
    <form
      className={isLoginFormVisible ? 'login-form' : 'login-form form-
hidden'}
      onSubmit={login}
    >
      <div className="control">
        <label htmlFor="email">Email</label>
        <input
          name='email'
          value={email}
          onChange={changeHandle}
          type="email"
          id="email"
          required
        />
      </div>
      <div className="control">
        <label htmlFor="password">Password</label>
        <input
          name='password'
          value={formData.password}
          onChange={changeHandle}
          type={showPassword ? "text" : "password"}
          id="password"
```

```
          required
        />
        <button
          type="button"
          onClick={togglePasswordVisibility}
          className="togglePasswordButton1"
        >
          <img
            src={showPassword ? eyeIcon:eyeSlashIcon }
            alt={showPassword ? "Hide password" : "Show password"}
            width="24"
            height="24"
          />
        </button>
      </div>
      <span style={{ color: "white" }}><input type="checkbox"
checked={rememberMe}
            onChange={(e) => setRememberMe(e.target.checked)} /> Remember
me</span>
      <div className="control">
      {isLoading ? (
              <PulseLoader color="#36d7b7" loading={isLoading} size={10} />
            ) : (
              <button type="submit" className="btn" disabled={isLoading}>
              {isLoading ? (
                <PulseLoader color="#fff" size={10} />
              ) :
                "Login"
              }
            </button>
            )}
    </div>
    <p>
      <Link to='/sign' onClick={() => toggleForm('login')}>Sign Up</Link> |
      <a href="#" onClick={() => toggleForm('lostPassword')}>Lost Your
Password?</a>
    </p>
    </form>
    {/* Lost Password Form */}
    <form
          className={isLostPasswordFormVisible ? 'lost-password-form' :
'lost-password-form form-hidden'}
          onSubmit={requestPasswordReset}
        >
          <h3>Lost Your Password?</h3>
          <h5>You will receive a link to create a new password via
email.</h5>
          <div className="form-group">
            <input
              type="email"
```

```
              placeholder="Email Address*"
              className="form-control"
              value={resetEmail}
              onChange={handleResetEmailChange}
              required
            />
          </div>
          {isLoading ? (
              <PulseLoader color="#36d7b7" loading={isLoading} size={10} />
            ) : (
              <button type="submit" className="submit-btn"
disabled={isLoading}>
              {isLoading ? (
                <PulseLoader color="#fff" size={10} />
              ) :
                "Reset"
              }
            </button>
            )}
          <p>
            <Link to='/login' className="login-btn" onClick={() =>
toggleForm('login')}>Log in</Link>
          </p>
        </form>
  </div>
  </div>
</div>
<ToastContainer />
</div>
```

); } export default Login;

< path="frontend/src/Movies/Movies.jsx"> import React, { useState, useEffect, useContext } from 'react'; import { Link, useNavigate } from 'react-router-dom'; import './Movies.css' import blankproduct from "../assets/nov.png"; import Navbar from '../Component/Navbar/Navbar'; import movie from './pics5/movieticket.jpg' import './Exit19.css' import { movieitems } from '../Data'; import Comment from '../Comments/Comment' import { ToastContainer, toast } from "react-toastify"; import { PulseLoader } from "react-spinners"; import "react-toastify/dist/ReactToastify.css"; import StarRate from '../Component/StarRating/StarRate'; import { MovieContext } from '../Context/MovieContext'; function Footer134() { return (

```
    <a href="https://www.facebook.com/"><i className="fab fa-facebook-
f"></i></a>
    <a href="https://twitter.com/LOGIN"><i className="fab fa-
twitter"></i></a>
    <a href="https://www.instagram.com/"><i className="fab fa-
instagram"></i></a>
    <a href="https://in.pinterest.com/"><i className="fab fa-
pinterest"></i></a>
```

```
  </div>
  <span style={{ color: "white" }}>ReelBite: Cinematic Cravings Movies
Page</span>
</footer>
```

); }

const Movies = () => { const navigate=useNavigate(); // const {addToCart, CartItems,handleItem,item} = useContext(MovieContext) const [movieItems, setMovieItems] = useState([]); const [searchTerm, setSearchTerm] = useState(''); const [exitIntent, setExitIntent] = useState(false); const[quantityAdd,setQuantityAdd] = useState(false); const [isLoading, setIsLoading] = useState(false); const handleProductClick = (productId) => { navigate(`/product/${productId}`); // Use navigate to go to the product details page }; useEffect(() => { const fetchMovies = async () => { try { const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/api/moviesproducts/all`); const data = await response.json(); setMovieItems(data); // Store fetched movie data in state } catch (error) { console.error('Error fetching movies:', error); } }; fetchMovies(); const handleMouseLeave = () => { setExitIntent(true); }; const handleMouseOut = (event) => { if (event.clientY < 0 && exitIntent) { document.getElementById('exit-popup11').style.display = 'block'; } }; document.addEventListener('mouseleave', handleMouseLeave); window.addEventListener('mouseout', handleMouseOut); return () => { document.removeEventListener('mouseleave', handleMouseLeave); window.removeEventListener('mouseout', handleMouseOut); }; }, [exitIntent]); const handleClosePopup = () => { document.getElementById('exit-popup11').style.display = 'none'; }; const filteredMovies = movieItems.filter((movie) => movie.productName.toLowerCase().includes(searchTerm) ); const addToCart = async (productId, quantity = 1) => { setQuantityAdd(false); const t=localStorage.getItem('auth-token'); if(!t){ toast.error('Failed to add Movie to cart:Login/Signup first'); return; } try { const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/api/cart/add`, { method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${localStorage.getItem('auth-token')}` // Add auth token if user is logged in }, body: JSON.stringify({ productId, quantity }) }); if (!response.ok) { toast.error('Failed to add item to cart'); throw new Error('Failed to add item to cart'); } const data = await response.json(); setQuantityAdd(true); toast.success('Movie added to cart successfully!'); } catch (error) { console.error('Error adding product to cart:', error); toast.error('Error adding product to cart'); } }; return ( <>

```
    <Navbar setSearchTerm={setSearchTerm} quantityAdded = {quantityAdd} />
     <div className='movieContainer'>
       <div className='movieitemdiv'>
      {filteredMovies.length > 0 ? (
        filteredMovies.map((data, index) => (
          <div key={index} className='movieCardCont'>
            <img className='movieCardImg' src={data.image} alt={data.name}
onClick={() => handleProductClick(data._id)} // Navigate to the product
details page on click
                style={{ cursor: "pointer" }} />
            <div className='movieCardRate'  onClick={() =>
```

```
handleProductClick(data._id)} // Navigate to the product details page on
click
                style={{ cursor: "pointer" }} >
              <span>{data.productName}</span>
              </div>
            <div className='movieCardRate' >
              <StarRate userId={localStorage.getItem("id")}
productId={data._id} productModel="movies"/>
            </div>
            <span>{data.category}</span><br />
            <span>${data.productPrice}</span>
            <span>Available: {data.available ? "Yes" : "No"}  </span>
            {data.available ? ( // Render button only if product is available
          <button className='com' onClick={() => addToCart(data._id)}>Buy
Tickets</button>
        ) : (
          <p style={{ color: 'red' }}>This product is currently
unavailable.</p>
        )}
            {/* <Comment productId={data._id} /> */}
          </div>
        ))
      ) : (
        <div className="blankCartContainer1">
              <img src={blankproduct} alt="Empty cart" />
              <div className="cartHeading1">
              </div>
            </div>)}
      </div>
    </div>
    <Footer134/>
        <div id="exit-popup11" style={{ display: 'none' }} >
        <h2>Don't Leave Yet!</h2>
        <h4>We have an exciting offer for you: </h4>
        <h4>Use Promocode 'TICKET100' to get 50% Cashback <br />on total
ticket price, upto Rs. 100.</h4>
        <img src={movie} alt="Offer" width="250" height="250" />
    <br />
    <button id='continue-button'onClick={handleClosePopup}>Continue</button>
  </div>
  </div>
  <ToastContainer />
</>
```

) } export default Movies

< path="frontend/src/Signup/Signup.jsx"> import React, { useState ,useEffect} from 'react'; import './Signup.css'; import main from './pics2/main.jpg'; import { Link, useNavigate } from 'react-router-dom'; import eyeIcon from "../assets/eye.png"; import eyeSlashIcon from "../assets/eye-2.png"; import { ToastContainer, toast } from "react-toastify"; import {

PulseLoader } from “react-spinners”; import “react-toastify/dist/ReactToastify.css”; function Signup() { const navigate = useNavigate(); const [isLoginFormVisible, setLoginFormVisible] = useState(true); const [isLoading, setIsLoading] = useState(false); const [isLostPasswordFormVisible, setLostPasswordFormVisible] = useState(false); const [formData, setFormData] = useState({ username: “”, email:””, password:”” }); const [showPassword, setShowPassword] = useState(false); const usernameRegex = /[1]{3,20}$/; const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z!/; const emailRegex = /[2]+@[^\s@]+.[^\s@]+$/;  const togglePasswordVisibility = () => { setShowPassword((prev) => !prev); // Toggle the password visibility  };  const [resetEmail, setResetEmail] = useState("");  const changeHandle = (e) => { setFormData({ ...formData, [e.target.name]: e.target.value });  };  const handleResetEmailChange = (e) => {  setResetEmail(e.target.value);  };  const signin1 = async (event) => {  event.preventDefault();  if (!usernameRegex.test(formData.username)) {  toast.error("Username must be between 3-20 characters and can only contain letters, numbers, and underscores.");  return;  } if (!emailRegex.test(formData.email)) {  toast.error("Please enter a valid email address."); return;  }  // Validate password  if (!passwordRegex.test(formData.password)) { toast.error("Password must be at least 8 characters, include an uppercase, lowercase, number, and special character.");  return;  }  try {  setIsLoading(true);  const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/api/user/register, { method: 'POST',  headers: {  Accept: 'application/json', 'Content-Type': 'application/json'  },  body: JSON.stringify(formData),  });  const responseData = await response.json();  setIsLoading(false);  if (responseData._id) { localStorage.setItem('auth-token', responseData.token); localStorage.setItem('id', responseData._id); localStorage.setItem('avatar', responseData.imageUrl|| "https://res.cloudinary.com/dwprhpk9r/image/upload/v1728546051/uploads/product_1728546048771.png.png");  toast.success("You are signed up... WELCOME TO ... !!");  setTimeout(() => { navigate('/home');  }, 2000);  } else { setIsLoading(false);  toast.error("Signup failed, please try again. " + responseData.error);  }  } catch (error) { setIsLoading(false);  toast.error("An error occurred during signup." + error);  console.error("Error during signup:", error);  }  }; const requestPasswordReset = async (event) => {  event.preventDefault(); try {  setIsLoading(true);  const response = await fetch(${import.meta.env.VITE_BACKEND_URL}/api/user/forgot-password`, { method: ‘POST’, headers: { Accept: ‘application/json’, ‘Content-Type’: ‘application/json’ }, body: JSON.stringify({ email: resetEmail }), }); const responseData = await response.json(); setIsLoading(false); if (responseData.success) { toast.success(‘Password reset successfully. Redirecting to login…’); setTimeout(() => { navigate(‘/login’); // Redirect to login page

---

[1] a-zA-Z0-9_

[2] ^

after a delay }, 1000); } else { toast.error(data.message || ‘Error resetting password’); } } catch (error) { setIsLoading(false); toast.error(‘An error occurred while resetting the password’); } }; const toggleForm = (formType) => { setLoginFormVisible(formType === ‘login’); setLostPasswordFormVisible(formType === ‘lostPassword’); }; return (

```
  <img className="bgimge" src={main} alt="pic" />
  <div className="login-page">
    <div className="box">
      <div className="form">
        <form
          className={isLoginFormVisible ? 'login-form' : 'login-form form-
hidden'}
          onSubmit={signin1}
        >
          <h3>Sign Up</h3>
          <div className="form-group">
            <input
              name='username'
              value={formData.username}
              onChange={changeHandle}
              type="text"
              placeholder="Name*"
              className="form-control"
              required
            />
          </div>
          <div className="form-group">
            <input
              name='email'
              value={formData.email}
              onChange={changeHandle}
              type="email"
              placeholder="Email Address*"
              className="form-control"
              required
            />
          </div>
          <div className="form-group">
            <input
              name='password'
              value={formData.password}
              onChange={changeHandle}
              type= {showPassword ? "text" : "password"}
              placeholder="Password*"
              className="form-control"
              required
            />
            <button
          type="button"
          onClick={togglePasswordVisibility}
```

```
          className="togglePasswordButton"
        >
          <img
            src={showPassword ? eyeIcon : eyeSlashIcon}
            alt={showPassword ? "Hide password" : "Show password"}
            width="24"
            height="24"
          />
        </button>
          </div>
          {isLoading ? (
              <PulseLoader color="#36d7b7" loading={isLoading} size={10} />
            ) : (
              <button type="submit" className="submit-btn"
disabled={isLoading}>
              {isLoading ? (
                <PulseLoader color="#fff" size={10} />
              ) :
                "Sign Up"
              }
            </button>
            )}
          <p>
            <Link to='/login' className="register-btn" onClick={() =>
toggleForm('login')}>Log in</Link> |
            <a href="#" className="lost-pass-btn" onClick={() =>
toggleForm('lostPassword')}>Lost Your Password?</a>
          </p>
        </form>
        {/* Lost Password Form */}
        <form
          className={isLostPasswordFormVisible ? 'lost-password-form' :
'lost-password-form form-hidden'}
          onSubmit={requestPasswordReset}
        >
          <h3>Lost Your Password?</h3>
          <h5>You will receive a link to create a new password via
email.</h5>
          <div className="form-group">
            <input
              type="email"
              placeholder="Email Address*"
              className="form-control"
              value={resetEmail}
              onChange={handleResetEmailChange}
              required
            />
          </div>
          {isLoading ? (
              <PulseLoader color="#36d7b7" loading={isLoading} size={10} />
```

```
            ) : (
              <button type="submit" className="submit-btn"
disabled={isLoading}>
              {isLoading ? (
                <PulseLoader color="#fff" size={10} />
              ) :
                "Reset"
              }
            </button>
            )}
          <p>
            <Link to='/login' className="login-btn" onClick={() =>
toggleForm('login')}>Log in</Link>
          </p>
        </form>
      </div>
    </div>
  </div>
  <ToastContainer />
</div>
```

); } export default Signup;